GO LANG API PROJECT — ALL SOURCE CODE
====================================
Combined from the uploaded golang_api.zip. Each section is labeled by role and original file path.
Note: This file is for reading/copying; copy each code section back into its corresponding .go file.

==
DB — db/postgres.go
==

```
package db

import (
	"database/sql"
	"fmt"
	"log"

	_ "github.com/jackc/pgx/v5/stdlib"
)

func Connect() *sql.DB{
	dsn:="postgres://postgres:aditya@localhost:5432/goapp"
	db,err:=sql.Open("pgx",dsn)

	if err!=nil{
		log.Fatal(err)
	}

	err=db.Ping();
	if err!=nil{
		log.Fatal(err)
	}
	fmt.Println("DB connected!!")
	return db
}
```

==
MODEL — models/user.go
==

```
package models

type User struct{
	Id int `json:"id"`
	Name string `json:"name"`
	Email string `json:"email"`
}
```

==
REPOSITORY — repository/user_repository.go
==

```
package repository

import (
	"database/sql"
	"golang-server/golang_api/models"
)

func GetUsers(db *sql.DB) ([]models.User,error){
```

```
	//fetching users from db &&  sending error if exist
	rows,err:=db.Query(
		"SELECT id,name,email from users",
	)


	if err!=nil{
		return nil,err
	}
	//closing reading of db

	defer rows.Close()


	//creating list of Users
	users:=[]models.User{}


	//taking input in u and appending in list
	for rows.Next(){
		var u models.User
		err=rows.Scan(&u.Id,&u.Name,&u.Email)
		if err!=nil{
			return nil,err
		}
		users=append(users, u)
	}

	//returning
	return users,rows.Err()
}

/*
db.Query() returns *sql.Rows, and row.Scan() is not valid for scanning a single
row directly.
Since you are fetching a user by ID, use db.QueryRow().


| Operation          | Method      | Purpose          |
|--------------------|-------------|------------------|
| Fetch one row      | `QueryRow()`| Get one user by ID |
| Fetch multiple rows| `Query()`   | Get all users    |
| Insert a row       | `Exec()`    | Create a user    |
| Update a row       | `Exec()`    | Edit a user      |
| Delete a row       | `Exec()`    | Delete a user    |

*/


//geting user by id
func GetUserById(db *sql.DB,id int) (models.User,error){
	var u models.User
	row:=db.QueryRow(
		"SELECT id,name,email FROM users WHERE id=$1",
		id,
	)
	err:=row.Scan(&u.Id,&u.Name,&u.Email)

	if err!=nil{
		return models.User{},err
	}
  return u,nil
}
```

```
func DeleteUserById(db *sql.DB,id int)(string,error){
      _,err:=db.Exec(
            "DELETE FROM users WHERE id=$1",id,
      )
      if err!=nil{
            return "User doest exist",err
      }

      return "Deleted succesfully",nil
}
func Create(db *sql.DB,u *models.User) error{

    err := db.QueryRow("INSERT INTO users (name, email) VALUES ($1, $2)
RETURNING id",
        u.Name,
        u.Email,
    ).Scan(&u.Id)

    return err
}


================================================================================
HANDLER — handlers/user_handler.go
================================================================================

package handlers

import (
      "database/sql"
      "encoding/json"
      "golang-server/golang_api/models"
      "golang-server/golang_api/repository"
      "net/http"
      "strconv"
)

type UserHandler struct{
      DB *sql.DB
}


func (h *UserHandler)GetUser(w http.ResponseWriter,r *http.Request){
      users,err:=repository.GetUsers(h.DB)
      if err!=nil{
            http.Error(w,"Failed  to fetchUser",http.StatusInternalServerError)
            return
      }

      w.Header().Set("Content-Type","Application/json")
      json.NewEncoder(w).Encode(users)
}


func (h *UserHandler)UserById(w http.ResponseWriter,r *http.Request){
      id,err:=strconv.Atoi(r.PathValue("id"))
      if err!=nil{
            http.Error(w,"Invlaid user Id",http.StatusBadRequest)
            return
      }

      user,err1:=repository.GetUserById(h.DB,id)
      if err1!=nil{
```

```
       http.Error(w,"failed to fetch user",http.StatusInternalServerError)
            return
      }
      w.Header().Set("Content-Type","Application/json")
      json.NewEncoder(w).Encode(user)
}


func (h *UserHandler)DeleteUser(w http.ResponseWriter,r *http.Request){
      str:=r.PathValue("id")
      id,err:=strconv.Atoi(str)

      if err!=nil{
            http.Error(w,"Invalid User id",http.StatusBadRequest)
            return
      }

      _,err1:=repository.DeleteUserById(h.DB,id)
      if err1!=nil{
            http.Error(w,"User Not Found",http.StatusNotFound)
            return
      }
      w.WriteHeader(http.StatusNoContent)
}

func (h *UserHandler)CreateUser(w http.ResponseWriter,r *http.Request){
      var user models.User

      err:=json.NewDecoder(r.Body).Decode(&user)
      if err!=nil{
            http.Error(w, "Invalid JSON", http.StatusBadRequest)
        return
      }

      if user.Name=="" || user.Email==""{
            http.Error(w,"Name and email is required",http.StatusBadRequest)
            return
      }

    err=repository.Create(h.DB,&user)

      if err!=nil{
            http.Error(w, "Failed to create
user",http.StatusInternalServerError)
        return
      }

      w.Header().Set("Content-Type", "application/json")
    w.WriteHeader(http.StatusCreated)
    json.NewEncoder(w).Encode(user)
}


==========================================================================
MAIN / ROUTES — main/main.go
==========================================================================

package main

import (
      "fmt"
      "golang-server/golang_api/db"
      "net/http"
      "golang-server/golang_api/handlers"
```

```
)

func main(){
  database:=db.Connect()
  defer database.Close()

  mux:=http.NewServeMux()

  UserHandler:=&handlers.UserHandler{
    DB:database,
  }

  mux.HandleFunc("GET /users",UserHandler.GetUser)
  mux.HandleFunc("GET /users/{id}",UserHandler.UserById)
  mux.HandleFunc("POST /users",UserHandler.CreateUser)
  mux.HandleFunc("DELETE /users/{id}",UserHandler.DeleteUser)

  server:=http.Server{
      Addr: ":3000",
      Handler: mux,
  }

  fmt.Println("Server is live at port 3000")
  server.ListenAndServe()
}
```